

# claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a87de686f5c808b1a.jsonl`  
- SHA-256: `a33aedbd79026a4109e442105ab233050391e4686a36ea1f6e8d38649faa8cca`  
- Source modified: `2026-06-10T16:30:23+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

## Transcript

### 1. user (2026-06-10T16:27:42.326Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: Savings oracle ignores available opening/closing bookends when the balance walk passes ? missing leading/trailing transactions persist as 'reconciled'  
SEVERITY CLAIMED: high  
FILE: src/muneem/parsers/savings.py (line 78-84, 194-205)  
DESCRIPTION: reconcile\_oracle returns True as soon as the per-row balance walk passes, never checking the printed opening/closing even when parse\_statement\_summary found them. The walk only verifies CONSECUTIVE rows that are present, so any omission at the statement's edges is invisible: a dropped first/last transaction line (the exact clusterer failure class seen in Axis May-2025 ? it was only quarantined because the dropped line was interior), a leading/trailing extract\_tables table that failed to map (parse\_savings silently keeps only oracle-endorsed tables, then re-checks the combined set whose walk still passes), or a truncated multi-page PDF. Empirically confirmed: with opening=10000.0 available, the oracle endorsed (a) a ledger whose entire leading half was missing (implied opening 9000 != 10000) and (b) a first row whose debit was misread as 9999 instead of 500 ? the first balance-bearing row's flows are never compared against anything. The HDFC parser has the same blind spot via its fallback: when parse\_summary returns None (e.g. the final page carrying the SUMMARY block is missing ? the very scenario that also truncates transactions), the verdict downgrades to reconcile(stmt).ok alone and grants RECONCILED. Note validate via verify() correctly withholds trust without bookends (test\_verify\_unavailable\_figures...), but neither the savings path nor HDFC's fallback uses that discipline.  
EVIDENCE QUOTED: savings.py:78-81: `def check(rows): if reconcile\_running\_balance([(r.debit, r.credit, r.balance) for r in rows]).ok: return True` ? short-circuits before the opening/closing check. hdfc.py:449-453: `ok = (validate\_against\_summary(stmt, summary).ok if summary is not None else reconcile(stmt).ok)`. Empirical run: oracle(opening=10000, closing=8500) endorsed rows starting at balance 8000 with a 1000 debit (implied opening 9000) -> True; endorsed first-row debit=9999 against balance 9500 -> True.  
SUGGESTED FIX: In reconcile\_oracle, when opening is available require the implied opening (rows[0].balance + debit0 - credit0 == opening) and when closing is available require rows[-1].balance == closing, in ADDITION to the walk (i.e. reuse verify()'s bookend checks). Treat walk-pass-without-any-bookend as a distinct lower-confidence verdict rather than 'reconciled'. For HDFC, treat an absent SUMMARY as UNRECONCILED (verify() already does this) instead of falling back to the walk alone.

Your job is to try to REFUTE it. Your lens: correctness: trace the actual code path the finding describes and check the claimed behavior is what the code really does (read the file and its callers/tests).

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

```
- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/,
```

testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).

- If you must inspect testdata/snapshots/\*.json or testdata/credentials/\*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client\_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service. Structured output only.

## 2. user (2026-06-10T16:27:48.533Z)

```
1      """Shared engine-driven savings-statement parsing (Axis, AU Bank, and future issuers).
2
3      Every issuer whose statement is a transaction table follows the same recipe, so it lives
here once
4      and each bank supplies only a :class:`~muneem.parsers.engine.Profile`:
5
6      1. Extraction proposes candidate cell-matrices ? one per ``extract_tables()`` table,
plus a
7      coordinate-clustered whole-statement matrix (for layouts ``extract_tables`` mis-
segments).
8      2. The generic engine maps each candidate's columns onto the canonical concepts.
9      3. Reconciliation disposes: a parse is trusted only if its per-row balance walks, or
? when a
10     vintage has no per-row balance ? the totals reconcile to the printed
``opening``/``closing``.
11
12     A statement splits its ledger across several extracted tables (page breaks), so we map
each table
13     on its own, keep the rows the oracle endorses, and re-check the combined set (which
catches a
14     cross-table balance break, i.e. a missing transaction). The result is oracle-gated:
a layout we
15     can't yet read returns ``verified=False`` and the caller persists nothing ? never
mislabeled rows.
16
17     No statement content is logged; only counts/booleans travel out.
18     """
19
20     from __future__ import annotations
21
22     import re
23     from dataclasses import dataclass, field
24     from pathlib import Path
25
26     import pdfplumber
27
28     from .clustering import cluster_to_matrix
29     from .concepts import ConceptRow, StatementConcepts
30     from .engine import Profile, map_table
31     from .validation import parse_amount, reconcile_running_balance, reconcile_total
32
33     _MONEY_TOKEN = re.compile(r"^\d{2}(\d{2})?(\d{2})?", re.IGNORECASE)
34     _OPENING_RE = re.compile(r"opening\s+balance[^\d-]*(-?\d{2})?", re.IGNORECASE)
35     _CLOSING_RE = re.compile(r"closing\s+balance[^\d-]*(-?\d{2})?", re.IGNORECASE)
36
37
38     @dataclass
39     class SavingsParse:
40         """Outcome of a savings parse: concept rows + whether reconciliation endorsed
them."""
41
42         account_number: str
```

```

43     rows: list[ConceptRow] = field(default_factory=list)
44     verified: bool = False
45
46
47     def parse_statement_summary(text: str) -> StatementConcepts:
48         """Read opening/closing balance from statement text (for the total-reconcile
oracle).
49
50         Best-effort regex; ``None`` when not found ? the oracle then relies on the per-row
walk.
51         """
52         opening = _OPENING_RE.search(text)
53         closing = _CLOSING_RE.search(text)
54         return StatementConcepts(
55             opening_balance=parse_amount(opening.group(1)) if opening else None,
56             closing_balance=parse_amount(closing.group(1)) if closing else None,
57         )
58
59
60     def txn_line_predicate(profile: Profile):
61         """A line is transaction-like if it carries one of the profile's dates *and* a money
token.
62
63         Used to restrict coordinate clustering to the transaction band (drops headers /
summaries /
64         address blocks that would otherwise wreck column inference).
65         """
66         patterns = profile.date_patterns
67
68         def keep(texts: list[str]) -> bool:
69             has_date = any(p.match(t) for p in patterns) for t in texts)
70             return has_date and any(_MONEY_TOKEN.match(t) for t in texts)
71
72         return keep
73
74
75     def reconcile_oracle(opening: float | None, closing: float | None):
76         """Trust a parse if the per-row balance walks, else if the totals reconcile to the
bookends."""
77
78         def check(rows: list[ConceptRow]) -> bool:
79             if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
80                 return True
81             if rows and opening is not None and closing is not None:
82                 return reconcile_total(rows, opening, closing)
83             return False
84
85         return check
86
87
88     def _reconstruct_by_balance_delta(
89         matrix: list[list[str]], profile: Profile, opening: float | None, tol: float = 0.01
90     ) -> list[ConceptRow]:
91         """Recover debit/credit from the running balance when columns won't split cleanly.
92
93         Needs only three things per row ? a date, the transaction amount, and the running
balance ? so
94         it tolerates messy column inference (a sliver column, an interleaved value-date
column, or a
95         single merged withdrawal/deposit column). Direction is the sign of the balance
change; the
96         caller's reconciliation then **cross-checks** that the amount equals |?balance| on
every row, so
97         a misread or a dropped row still fails (it is *not* circular). Needs the printed
opening balance

```

```

98         to fix the first row's direction; returns ``[]`` if it can't form a clean series.
99         ""
100         patterns = profile.date_patterns or (re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),)
101         rows = [(str(c).strip() if c else "") for c in r] for r in (matrix or []) if r]
102         if not rows or opening is None:
103             return []
104         ncols = max(len(r) for r in rows)
105
106         def is_date(s: str) -> bool:
107             return any(p.match(s) for p in patterns)
108
109         def money(s: str) -> float | None:
110             return parse_amount(s) if _MONEY_TOKEN.match(s) else None
111
112         # date column = the leftmost of the (first few) columns with the most date cells
113         date_counts: dict[int, int] = {}
114         for r in rows:
115             for c in range(min(ncols, 3)):
116                 if c < len(r) and is_date(r[c]):
117                     date_counts[c] = date_counts.get(c, 0) + 1
118         if not date_counts:
119             return []
120         top = max(date_counts.values())
121         date_col = min(c for c, n in date_counts.items() if n == top)
122
123         txn_rows = [r for r in rows if date_col < len(r) and is_date(r[date_col])]
124         if len(txn_rows) < 2:
125             return []
126
127         # A money column's money cells must outnumber its date cells ? so a sparsely-filled
deposit
128         # column still counts, but an interleaved value-date column (mostly dates) does not.
The
129         # running balance is the rightmost money column; the rest are the flow
(withdrawal/deposit).
130         money_cols = []
131         for c in range(ncols):
132             m = sum(1 for r in txn_rows if c < len(r) and money(r[c]) is not None)
133             d = sum(1 for r in txn_rows if c < len(r) and is_date(r[c]))
134             if m >= 1 and m > d:
135                 money_cols.append(c)
136         if len(money_cols) < 2:
137             return []
138         balance_col = max(money_cols)
139         flow_cols = [c for c in money_cols if c != balance_col]
140
141         text_cols = [c for c in range(ncols) if c != date_col and c not in money_cols]
142         narr_col = (
143             max(text_cols, key=lambda c: sum(len(r[c]) for r in txn_rows if c < len(r)))
144             if text_cols
145             else None
146         )
147
148         out: list[ConceptRow] = []
149         prev = opening
150         for r in txn_rows:
151             bal = money(r[balance_col]) if balance_col < len(r) else None
152             if bal is None:
153                 return [] # a running balance on every row is required for the delta
154             amt = next((money(r[c]) for c in flow_cols if c < len(r) and money(r[c]) is not
None), None)
155             debit = credit = None
156             if amt is not None:
157                 if bal < prev - tol:
158                     debit = amt

```

```

159         elif bal > prev + tol:
160             credit = amt
161         else:
162             debit = amt # zero net with an amount present ? reconciliation will
flag it
163         out.append(
164             ConceptRow(
165                 txn_date=r[date_col],
166                 narration=(r[narr_col] if narr_col is not None and narr_col < len(r)
else ""),
167                 debit=debit,
168                 credit=credit,
169                 balance=bal,
170             )
171         )
172         prev = bal
173     return out
174
175
176     def parse_savings(pdf_path: Path, profile: Profile, password: str | None = None) ->
SavingsParse:
177         """Parse a savings statement into concept rows + a reconciliation verdict, per the
profile.
178
179         Per-table mapping first (each extracted table mapped on its own; oracle-endorsed
rows combined
180         and re-checked), then a coordinate-clustering fallback. Returns ``verified=False``
if neither
181         reconciles, so the caller can refuse to persist.
182         """
183         with pdfplumber.open(pdf_path, password=password) as pdf:
184             words_by_page = [
185                 page.extract_words(use_text_flow=False, keep_blank_chars=False) for page in
pdf.pages
186             ]
187             tables = [t for page in pdf.pages for t in (page.extract_tables() or [])]
188             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
189
190             account_number = profile.first_account(head) or "UNKNOWN"
191             summary = parse_statement_summary(head)
192             oracle = reconcile_oracle(summary.opening_balance, summary.closing_balance)
193
194             confirmed: list[ConceptRow] = []
195             for table in tables:
196                 res = map_table(table, profile, oracle=oracle)
197                 if res and res.verified:
198                     confirmed.extend(res.rows)
199             if confirmed and oracle(confirmed):
200                 return SavingsParse(account_number, confirmed, True)
201
202             matrix = cluster_to_matrix(words_by_page, keep_line=txn_line_predicate(profile))
203             clustered = map_table(matrix, profile, oracle=oracle)
204             if clustered and clustered.verified:
205                 return SavingsParse(account_number, clustered.rows, True)
206
207             # 3) balance-delta reconstruction ? recovers vintages whose columns don't split into
a clean
208             #     debit/credit (merged amount column, interleaved value-date, sliver columns):
derive the
209             #     direction from the running balance and let the oracle cross-check amount ==
|?balance|.
210             delta = _reconstruct_by_balance_delta(matrix, profile, summary.opening_balance)
211             if delta and oracle(delta):
212                 return SavingsParse(account_number, delta, True)

```

```

213
214         fallback = confirmed or (clustered.rows if clustered else [])
215         return SavingsParse(account_number, fallback, False)
216
217
218     __all__ = [
219         "SavingsParse",
220         "parse_savings",
221         "parse_statement_summary",
222         "txn_line_predicate",
223         "reconcile_oracle",
224     ]
225

```

### 3. user (2026-06-10T16:27:49.162Z)

```

src\muneem\parsers\__init__.py
src\muneem\parsers\concepts.py
src\muneem\parsers\validation.py
src\muneem\parsers\clustering.py
src\muneem\parsers\profiles.py
src\muneem\parsers\savings.py
src\muneem\parsers\engine.py
src\muneem\parsers\base.py
src\muneem\parsers\au.py
src\muneem\parsers\axis.py
src\muneem\parsers\hdfc.py

```

### 4. user (2026-06-10T16:27:56.341Z)

```

1     """The verifier ? reconciliation as a layout-independent truth oracle.
2
3     Reconciliation proves a parse is complete and correct without inspecting any sensitive
value**:
4     if a transaction is missed or misread, the arithmetic stops adding up. That is what lets
us trust a
5     parse of a statement we never eyeball, and (later) what makes a probabilistic extractor
safe on
6     money ? extraction proposes, reconciliation disposes (see
7     [docs/parser-architecture.md] (../..../docs/parser-architecture.md) $2, $5).
8
9     This module grew from a single running-balance walk into the verifier the architecture
calls for:
10
11     * per-row walk ? reconcile_running_balance: balance[i] == balance[i-1] ? debit
+ credit.
12     * total reconcile ? reconcile_total: opening + ?credit ? ?debit == closing
(work even
13     with no per-row balance, e.g. Axis savings).
14     * bookend ? verify: parsed Dr/Cr counts + opening/closing vs the bank's printed
SUMMARY.
15     * selection ? select_reconciling: among candidate concept mappings, pick the one
arithmetic
16     endorses (the seam the L2 mapping search plugs into).
17     * confidence ? Confidence + BookendReport.confidence: a recorded verdict,
not an
18     assumption.
19
20     Reports carry counts / booleans / a verdict only ? never an amount.
21     """
22
23     from __future__ import annotations
24
25     import re
26     from collections.abc import Sequence

```

```

27 from dataclasses import dataclass
28 from enum import StrEnum
29
30 from .concepts import ConceptRow, StatementConcepts
31
32 _CR_DR_SUFFIX = re.compile(r"(?i)\s*(cr|dr)\s*$")
33
34
35 def parse_amount(value: object) -> float | None:
36     """Parse a statement money cell to a float.
37
38     Returns ``None`` only for a genuinely absent cell (None / empty / ``"-`` /
39     non-numeric); ``"0.00"`` parses to ``0.0``. This deliberately distinguishes
40     *absent* from *zero* ? unlike ``base.safe_amount``, which maps ``"0.00"`` to
41     ``None`` and would corrupt a real zero balance and the reconciliation math.
42     """
43     if value is None:
44         return None
45     text = _CR_DR_SUFFIX.sub("", str(value).strip()).replace(",", "").strip()
46     if text in ("", "-"):
47         return None
48     try:
49         return float(text)
50     except ValueError:
51         return None
52
53
54 @dataclass
55 class ReconcileReport:
56     """Result of a running-balance walk. Counts only, never an amount."""
57
58     n: int # rows considered
59     compared: int # consecutive (balance-bearing) pairs actually checked
60     breaks: int
61     first_break: int | None = None
62
63     @property
64     def ok(self) -> bool:
65         return self.compared > 0 and self.breaks == 0
66
67
68 def reconcile_running_balance(
69     entries: Sequence[tuple[float | None, float | None, float | None]],
70     tol: float = 0.01,
71 ) -> ReconcileReport:
72     """Verify ``balance[i] == balance[i-1] - debit[i] + credit[i]`` across rows.
73
74     ``entries`` is an ordered list of ``(debit, credit, balance)``; ``debit`` and
75     ``credit`` may be ``None`` (treated as 0.0). A row whose ``balance`` is ``None``
76     is skipped (can't anchor on it). Because the walk spans the whole statement, a
77     clean result also rules out any row missed *between* two balances ? which is the
78     failure mode silent ``extract_tables()`` row-drops would otherwise hide.
79     """
80     breaks = 0
81     first_break: int | None = None
82     compared = 0
83     prev: float | None = None
84     for i, (debit, credit, balance) in enumerate(entries):
85         if balance is None:
86             continue
87         if prev is not None:
88             expected = prev - (debit or 0.0) + (credit or 0.0)
89             compared += 1
90             if abs(expected - balance) > tol:
91                 breaks += 1

```

```

92         if first_break is None:
93             first_break = i
94         prev = balance
95         return ReconcileReport(
96             n=len(entries), compared=compared, breaks=breaks, first_break=first_break
97         )
98
99
100     def _row_entries(
101         rows: Sequence[ConceptRow],
102     ) -> list[tuple[float | None, float | None, float | None]]:
103         """Project concept rows onto the ``(debit, credit, balance)`` tuples the walk
104         consumes."""
105         return [(r.debit, r.credit, r.balance) for r in rows]
106
107     def reconcile_total(
108         rows: Sequence[ConceptRow], opening: float, closing: float, tol: float = 0.01
109     ) -> bool:
110         """Verify the layout-independent identity ``opening + ?credit ? ?debit == closing``.
111
112         This is the oracle for statements with no reliable per-row balance (e.g. Axis
113         savings):
114         it checks the whole statement against its own opening/closing bookends without
115         needing each
116         row's running balance. ``opening`` and ``closing`` come from the bank's
117         SUMMARY/header
118         (``StatementConcepts``); the caller must have them.
119         """
120         delta = sum(r.credit or 0.0 for r in rows) - sum(r.debit or 0.0 for r in rows)
121         return abs((opening + delta) - closing) <= tol
122
123     class Confidence(StrEnum):
124         """Recorded trust verdict for a parsed statement (mirrors ``documents.status``
125         values).
126
127         ``RECONCILED`` ? the parse passed every available check, so it is trusted.
128         ``UNRECONCILED`` ?
129         a check failed (or none could run); the parse is suspect and is quarantined rather
130         than trusted.
131         """
132         RECONCILED = "reconciled"
133         UNRECONCILED = "unreconciled"
134
135     @dataclass
136     class BookendReport:
137         """A parse cross-checked against the bank's own SUMMARY. Booleans + counts only.
138
139         ``None`` for a check means the figure wasn't available to verify (not a failure).
140         ``ok``
141         requires the per-row balance walk plus every available check to pass.
142         """
143         n: int
144         reconciles: bool
145         debit_count_ok: bool | None = None
146         credit_count_ok: bool | None = None
147         opening_ok: bool | None = None
148         closing_ok: bool | None = None
149
150         @property
151         def ok(self) -> bool:

```



```

149         checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok,
self.closing_ok]
150         return self.n > 0 and self.reconciles and all(c is True for c in checks)
151
152     @property
153     def confidence(self) -> Confidence:
154         return Confidence.RECONCILED if self.ok else Confidence.UNRECONCILED
155
156
157     def verify(
158         rows: Sequence[ConceptRow], concepts: StatementConcepts, tol: float = 0.01
159     ) -> BookendReport:
160         """Bookend a parse against the bank's printed SUMMARY: reconciliation + the printed
figures.
161
162         The per-row walk catches any mid-statement error/omission; the Dr/Cr counts catch
any
163         extra/missing row; the opening & closing balances catch a missing first/last row.
Each printed
164         figure is only checked when present (``None`` ? not checked). The opening balance is
checked by
165         implication (``balance[0] ? credit[0] + debit[0]`` should equal the printed
opening).
166         """
167         reconciles = reconcile_running_balance(_row_entries(rows), tol).ok
168         debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
169         if concepts.debit_count is not None:
170             debit_count_ok = sum(1 for r in rows if (r.debit or 0.0) > 0) ==
concepts.debit_count
171         if concepts.credit_count is not None:
172             credit_count_ok = sum(1 for r in rows if (r.credit or 0.0) > 0) ==
concepts.credit_count
173         if rows and rows[-1].balance is not None and concepts.closing_balance is not None:
174             closing_ok = abs(rows[-1].balance - concepts.closing_balance) <= tol
175         if rows and rows[0].balance is not None and concepts.opening_balance is not None:
176             implied_opening = rows[0].balance - (rows[0].credit or 0.0) + (rows[0].debit or
0.0)
177             opening_ok = abs(implied_opening - concepts.opening_balance) <= tol
178         return BookendReport(
179             n=len(rows),
180             reconciles=reconciles,
181             debit_count_ok=debit_count_ok,
182             credit_count_ok=credit_count_ok,
183             opening_ok=opening_ok,
184             closing_ok=closing_ok,
185         )
186
187
188     def select_reconciling(candidates: Sequence[Sequence[ConceptRow]], tol: float = 0.01) ->
int | None:
189         """Among candidate concept-row mappings, return the index of the first that
reconciles.
190
191         This is the *selection* half of "extraction proposes, reconciliation disposes": when
the
192         extraction ladder can't be sure which column?concept assignment is right, it
proposes several
193         and lets the per-row walk pick the correct one. Returns ``None`` if none reconcile.
194
195         The candidate *generation* (the L2 mapping search) is Step 2 of the architecture
plan; this is
196         the verifier-side seam it plugs into. Selection here uses the per-row walk, so
candidates need a
197         running balance ? balance-less statements select via :func:`reconcile_total` at the
call site.

```

```

198         """
199         for i, rows in enumerate(candidates):
200             if reconcile_running_balance(_row_entries(rows), tol).ok:
201                 return i
202         return None
203
204
205     __all__ = [
206         "parse_amount",
207         "ReconcileReport",
208         "reconcile_running_balance",
209         "reconcile_total",
210         "Confidence",
211         "BookendReport",
212         "verify",
213         "select_reconciling",
214     ]
215

```

## 5. user (2026-06-10T16:27:57.052Z)

```

400         )
401         return out
402
403
404     @register_parser
405     class HDFCParser(StatementParser):
406         """Registered parser for HDFC savings / transaction-account statements."""
407
408         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
409             low = (text or "").lower()
410             is_hdfc = "hdfc bank" in low or "hdfcbank" in low or "hdfcbank" in (sender or
"").lower()
411             is_statement = "savings account details" in low or (
412                 "narration" in low and "closing balance" in low
413             )
414             return is_hdfc and is_statement
415
416         def parse_and_insert(
417             self,
418             pdf_path: Path,
419             conn,
420             doc_id: int,
421             password: str | None = None,
422         ) -> int:
423             """Parse, insert rows into ``hdfc_transactions``, and record parse quality.
424
425             Sets ``documents.status`` to 'reconciled' / 'unreconciled' based on the
426             balance walk (and the bank's SUMMARY when present) so a downstream consumer
427             can tell a trustworthy parse from a suspect one. Returns the row count.
428             """
429             try:
430                 stmt = parse_statement(pdf_path, password=password or "")
431             except ParseError:
432                 raise
433             except Exception as exc:
434                 raise ParseError(f"failed to parse HDFC statement
{redact_filename(pdf_path)}") from exc
435             for t in stmt.transactions:
436                 dbmod.insert_hdfc_transaction(
437                     conn,
438                     document_id=doc_id,
439                     account_number=stmt.account_number,
440                     txn_date=t.date,

```

```

441         value_date=t.value_date,
442         particulars=t.narration,
443         ref=t.ref,
444         withdrawal=t.withdrawal,
445         deposit=t.deposit,
446         closing_balance=t.closing_balance,
447     )
448     summary = parse_summary(pdf_path, password=password or "")
449     ok = (
450         validate_against_summary(stmt, summary).ok
451         if summary is not None
452         else reconcile(stmt).ok
453     )
454     verdict = Confidence.RECONCILED if ok else Confidence.UNRECONCILED
455     dbmod.record_parse(
456         conn, doc_id, confidence=verdict.value, parser="HDFCParser",
457         parser_version=__version__
458     )
458     return len(stmt.transactions)
459

```

## 6. user (2026-06-10T16:28:29.699Z)

```

1     """Axis Bank statement parsers: savings/transaction account + credit card.
2
3     **Savings runs on the shared engine-driven path** (:mod:`muneem.parsers.savings`): per-
4     table +
5     coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
6     gated** so a
7     vintage it can't read stores nothing (`unreconciled`) rather than mislabeled rows.
8     Axis specifics
9     live entirely in `AXIS_PROFILE`; this replaced the old fixed-column `extract_tables`
10    mapping.
11
12    Credit-card statements carry no running balance, so `AxisCreditCardParser` can't
13    reconcile
14    (parse/persist split + count only). No statement content is logged here.
15    """
16
17    from __future__ import annotations
18
19    import re
20    import sqlite3
21    from dataclasses import dataclass
22    from pathlib import Path
23
24    import pdfplumber
25
26    from muneem import __version__
27    from muneem import db as dbmod
28    from muneem.errors import ParseError
29    from muneem.redaction import redact_filename
30
31    from . import register_parser
32    from .base import StatementParser
33    from .profiles import AXIS_PROFILE
34    from .savings import SavingsParse, parse_savings, parse_statement_summary,
35    txn_line_predicate
36    from .validation import Confidence, parse_amount
37
38    _CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}") # Axis credit card: dd/mm/yyyy
39    _CARD_RES = [re.compile(r"(\d{4}XXXX\d{4})"), re.compile(r"(\d{6}\*\d{4})")]
40
41    def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:

```

```

37         for rx in patterns:
38             m = rx.search(text)
39             if m:
40                 return m.group(1).strip()
41         return None
42
43
44     # --- Savings / transaction account (concept core)
-----
45
46
47     # Back-compat names; the savings logic now lives in muneem.parsers.savings (shared with
AU Bank).
48     AxisSavings = SavingsParse
49     parse_axis_summary = parse_statement_summary
50     _is_txn_line = txn_line_predicate(AxisPROFILE)
51
52
53     def parse_axis_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
54         """Parse an Axis savings statement via the shared engine-driven path
(savings.parse_savings)."""
55         return parse_savings(pdf_path, AxisPROFILE, password)
56
57
58     @register_parser
59     class AxisParser(StatementParser):
60         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
61             if sender and "axisbank.com" in sender.lower():
62                 return True
63             t = text or ""
64             return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)
65
66     def parse_and_insert(
67         self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
68     ) -> int:
69         """Persist a savings parse **only if it reconciles**; otherwise store nothing.
70
71         Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
72         parser can't yet read surfaces loudly instead of saving mislabeled rows.
73         """
74         try:
75             parsed = parse_axis_savings(pdf_path, password)
76         except ParseError:
77             raise
78         except Exception as exc:
79             # Filename only ? never statement text; original cause kept via `from`.
80             name = redact_filename(pdf_path)
81             raise ParseError(f"failed to parse Axis statement {name}") from exc
82
83         if not parsed.verified:
84             dbmod.record_parse(
85                 conn,
86                 doc_id,
87                 confidence=Confidence.UNRECONCILED.value,
88                 parser="AxisParser",
89                 parser_version=__version__,
90             )
91             return 0
92
93         for r in parsed.rows:
94             dbmod.insert_axis_transaction(

```

```

95         conn,
96         document_id=doc_id,
97         account_number=parsed.account_number,
98         txn_date=r.txn_date,
99         value_date=r.value_date or "",
100        particulars=r.narration,
101        debit=r.debit,
102        credit=r.credit,
103        balance=r.balance,
104    )
105    dbmod.record_parse(
106        conn,
107        doc_id,
108        confidence=Confidence.RECONCILED.value,
109        parser="AxisParser",
110        parser_version=__version__,
111    )
112    return len(parsed.rows)
113
114
115    # --- Credit card (no running balance; count only)
116    -----
117
118    @dataclass
119    class AxisCCTxn:
120        txn_date: str
121        particulars: str
122        merchant_category: str
123        amount: float
124        type: str # "Cr" or "Dr"
125
126
127    def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
128        """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
129        rows: list[AxisCCTxn] = []
130        for table in tables or []:
131            for row in table or []:
132                if not row or len(row) < 9:
133                    continue
134                date_str = str(row[0]).strip() if row[0] else ""
135                if not _CC_DATE_RE.match(date_str):
136                    continue
137                amount_str = str(row[8]).strip() if row[8] else ""
138                is_cr = "Cr" in amount_str
139                amount = parse_amount(re.sub(r"^\d.", "", amount_str)) or 0.0
140                rows.append(
141                    AxisCCTxn(
142                        txn_date=date_str,
143                        particulars=str(row[2]).strip() if row[2] else "",
144                        merchant_category=str(row[7]).strip() if row[7] else "",
145                        amount=amount,
146                        type="Cr" if is_cr else "Dr",
147                    )
148                )
149        return rows
150
151
152    def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str,
list[AxisCCTxn]]:
153        """Open an Axis credit-card statement and return (card_number, transactions)."""
154        card_number = "UNKNOWN"
155        rows: list[AxisCCTxn] = []
156        with pdfplumber.open(pdf_path, password=password) as pdf:
157            head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,

```

```

len(pdf.pages)))
158         card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
159         for page in pdf.pages:
160             rows.extend(_rows_from_cc_tables(page.extract_tables()))
161         return card_number, rows
162
163
164     @register_parser
165     class AxisCreditCardParser(StatementParser):
166         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
167             return "Axis Bank" in text and "Credit Card Statement" in text
168
169         def parse_and_insert(
170             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
171         ) -> int:
172             try:
173                 card_number, rows = parse_axis_cc(pdf_path, password)
174             except ParseError:
175                 raise
176             except Exception as exc:
177                 raise ParseError(
178                     f"failed to parse Axis CC statement {redact_filename(pdf_path)}"
179                 ) from exc
180             for t in rows:
181                 dbmod.insert_axis_cc_transaction(
182                     conn,
183                     document_id=doc_id,
184                     account_number=card_number,
185                     txn_date=t.txn_date,
186                     particulars=t.particulars,
187                     merchant_category=t.merchant_category,
188                     amount=t.amount,
189                     type=t.type,
190                 )
191             # No running balance to reconcile; documents.status stays as the caller set it.
192
193             # record provenance (with no confidence verdict).
194             dbmod.record_provenance(
195                 conn,
196                 document_id=doc_id,
197                 source_type="pdf_statement",
198                 parser="AxisCreditCardParser",
199                 parser_version=__version__,
200                 mapping_layer="deterministic",
201             )
202             return len(rows)

```

## 7. user (2026-06-10T16:28:30.955Z)

```

267: def parse_statement(pdf_path: Path | str, password: str = "") -> Statement:
268-     """Parse an HDFC statement PDF into a :class:`Statement` of transactions.
269-
270-     ``password`` is forwarded to pdfplumber for encrypted statements and used
271-     in-memory only ? never stored or logged.
272-
273-     """
274-     path = Path(pdf_path)
275-     transactions: list[Transaction] = []
276-     account_number: str | None = None
277-     with pdfplumber.open(str(path), password=password or "") as pdf:
278-         for page_no, page in enumerate(pdf.pages):
279-             if account_number is None and page_no < 2:
280-                 account_number = _extract_account_number(page.extract_text() or "")

```

```

280-         words = page.extract_words(use_text_flow=False, keep_blank_chars=False)
281-         body = _table_body(_group_lines(words))
282-         transactions.extend(extract_transactions_from_lines(body, page_no))
283-     return Statement(path=path, transactions=transactions, account_number=account_number)
284-
285-
286-def reconcile(statement: Statement, tol: float = 0.01) -> ReconcileReport:
287-     """Check the running balance via the shared reconciler (closing[i] ==
288-     closing[i-1] - withdrawal + deposit).
289-
290-     Spans page breaks (transactions are in document order), so a clean report also
291-     validates page stitching. Reports counts only ? no amounts.
292-     """
293-     entries = [(t.withdrawal, t.deposit, t.closing_balance) for t in statement.transactions]
294-     return reconcile_running_balance(entries, tol)
295-
296-
297-def _parse_summary_lines(lines: list[str]) -> StatementSummary | None:
298-     """Pure core: read HDFC's SUMMARY block from extracted text lines.
299-
300-     Layout (stable across statements)::
301-
302-         SUMMARY
303-         Opening Balance  Debit Amount  Credit Amount  Closing Balance
304-         <opening>        <debitAmt>   <creditAmt>   <closing>
305-         Debit Count     Credit Count
306-         <debitCount> <creditCount>
307-     """
308-
309-
310-
311-
312-
313-
314-
315-
316-
317-
318-
319-
320-
321-
322-
323-
324-
325-
326-
327-
328-
329-
330-
331-
332-
333-
334-
335-
336-
337-
338-
339-
340-
341-def parse_summary(pdf_path: Path | str, password: str = "") -> StatementSummary | None:
342-     """Read the bank's own SUMMARY figures from the statement, or None if absent."""
343-     with pdfplumber.open(str(Path(pdf_path)), password=password or "") as pdf:
344-         for page in pdf.pages:
345-             text = page.extract_text() or ""
346-             if "SUMMARY" in text:
347-                 summary = _parse_summary_lines(text.splitlines())
348-                 if summary is not None:
349-                     return summary
350-     return None
351-
352-
353-def validate_against_summary(
354-     statement: Statement, summary: StatementSummary, tol: float = 0.01
355-) -> ValidationReport:
356-     """Cross-check a parse against the bank's SUMMARY: reconciliation + bookends.
357-
358-     Maps HDFC's rows onto the canonical concepts and delegates to the shared verifier
359-     (:func:`muneem.parsers.validation.verify`): reconciliation catches any mid-statement
360-     error/omission, the Dr/Cr counts catch any extra/missing row, and the opening & closing
361-     balances catch a missing first/last row.
362-     """
363-     rows = [t.to_concept() for t in statement.transactions]
364-     return verify(rows, summary, tol)
365-
366-
367-def write_csv(statement: Statement, out_path: Path | str) -> Path:
368-     """Write parsed transactions to CSV for local QA (the 'export to a sheet' idea).
369-
370-     Writes real data ? call only for a local, gitignored destination.
371-     """
372-     out = Path(out_path)
373-     out.parent.mkdir(parents=True, exist_ok=True)

```

## 8. assistant (2026-06-10T16:28:44.394Z)

All static claims check out so far. Let me verify the empirical claim by running the oracle

directly, check the AU caller, and the referenced test.

#### 9. user (2026-06-10T16:28:54.065Z)

Exit code 1

Traceback (most recent call last):

```
File "<string>", line 2, in <module>
    from muneem.parsers.savings import reconcile_oracle
ModuleNotFoundError: No module named 'muneem'
```

#### 10. user (2026-06-10T16:28:55.251Z)

```
tests\parsers\test_validation.py:87:def
test_verify_unavailable_figures_are_not_checked_so_trust_is_withheld():
tests\parsers\test_validation.py-88-    # No SUMMARY figures: the per-row walk still passes, but
with nothing to bookend against the
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-89-    # parse is not *trusted*
(ok=False). This is why HDFC falls back to reconcile() when absent.
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-90-    r =
verify(_concept_rows(), StatementConcepts())
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-91-    assert r.reconciles
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-92-    assert r.debit_count_ok
is None and r.credit_count_ok is None
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-93-    assert r.opening_ok is
None and r.closing_ok is None
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-94-    assert not r.ok and
r.confidence is Confidence.UNRECONCILED
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-95-
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-96-
tests\parsers\test_validation.py-97-def test_verify_catches_wrong_count():
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-98-    concepts =
StatementConcepts(
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-99-    opening_balance=10000.0, closing_balance=9999.5, debit_count=1, credit_count=1
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-100-    )
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-101-    r =
verify(_concept_rows(), concepts) # there are 2 debits, not 1
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-102-    assert r.reconciles and
r.debit_count_ok is False and not r.ok
```

#### 11. user (2026-06-10T16:29:02.363Z)

```
CLAUDE.md
DESIGN.md
Makefile
README.md
docs
pyproject.toml
src
testdata
tests
tools
Activate.ps1
__pycache__
activate
activate.bat
activate.fish
```

#### 12. user (2026-06-10T16:29:14.727Z)

```
missing-leading-half endorsed: True
bad-first-debit endorsed: True
missing-trailing endorsed: True
verify lead: reconciles=True opening_ok=False closing_ok=True ok=False
```



verify first: reconciles=True opening\_ok=False closing\_ok=True ok=False  
verify tail: reconciles=True opening\_ok=True closing\_ok=False ok=False

### 13. assistant (2026-06-10T16:29:33.283Z)

All three empirical claims reproduce exactly. Let me verify the remaining contextual claims ? the AU caller pattern and the Axis May-2025 quarantine reference.

### 14. user (2026-06-10T16:29:35.280Z)

```
48-         raise ParseError(f"failed to parse AU statement {redact_filename(pdf_path)}")
from exc
49-
50-     if not parsed.verified:
51-         dbmod.record_parse(
52-             conn,
53-             --
54-             conn,
55-             doc_id,
56-             confidence=Confidence.RECONCILED.value,
57-             parser="AUParser",
58-             parser_version=__version__,
```

### 15. user (2026-06-10T16:29:36.428Z)

```
docs\CURRENT_STATE.md:30:[Omitted long matching line]
docs\CURRENT_STATE.md:55:* **Recover the May 2025 vintage** : the clusterer drops one transaction
line ? incomplete ledger; needs smarter line recovery. Isolated and safe (it currently stores
nothing; June 2025 was recovered via balance-delta reconstruction).
docs\parser-architecture.md:93:- **Confidence verdict:** `reconciled` (per-row or total) +
`bookended` ? trust; else quarantine.
docs\parser-architecture.md:110:- reconciliation breaks, or parsed counts ? printed summary ?
**quarantine** (store nothing or store
docs\parser-architecture.md:198:- **Confidence policy** ? what does the CLI/ledger do with a
`quarantined` statement (skip, show with
docs\ROADMAP.md:36:> **Remaining:** recover the dropped line (smarter line recovery in the
clusterer) for May 2025 and
docs\ROADMAP.md:48:2. **Axis May 2025 recovery** ? recover the one transaction line the
clusterer drops (incomplete
tests\parsers\test_savings.py:6:(a balance jump matching no single amount) which must NOT
reconcile (the real Axis May 2025 case).
tests\parsers\test_savings.py:41:    # so the reconstruction must NOT reconcile ? the oracle
quarantines an incomplete ledger (May).
src\muneeem\parsers\validation.py:125:    a check failed (or none could run); the parse is
suspect and is quarantined rather than trusted.
```

### 16. user (2026-06-10T16:30:17.088Z)

Structured output provided successfully

### 17. assistant (2026-06-10T16:30:23.256Z)

Verification complete.